Exploratory Data Analysis project

JESWANTH G
23BDS0212

Drive :-
https://drive.google.com/file/d/1CiGNwyqMdqT1ZIe2iqrDSnM-aLy5UIqa/view?usp=sharing

```
!pip install -q adjustText

import os, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

from scipy import stats
from scipy.stats import skew, kurtosis
from scipy.spatial.distance import pdist
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

import statsmodels.api as sm
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

from adjustText import adjust_text
from pandas.plotting import parallel_coordinates
```

```
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)

DATA = "BEPS.csv"
if not os.path.exists(DATA):
    from google.colab import files
    uploaded = files.upload()
    DATA = list(uploaded.keys())[0]

NUM_COLS = ['age', 'economic.cond.national',
'economic.cond.household',
            'Blair', 'Hague', 'Kennedy', 'Europe', 'political.knowledge']

print("Setup done. Data file:", DATA)


print("\n\n==================== DA 1
====================")

dataframe1 = pd.DataFrame({
    'ProductID': range(101, 111),
    'NameofProd': ['Laptop', 'Mouse', 'Keyboard', 'Monitor',
'Printer',
                   'Scanner', 'Webcam', 'Headset', 'Router', 'Tablet'],
    'Price': [55000, 500, 1200, 9000, 7500, 6500, 2200, 1800, 2500,
18000]
})

dataframe2 = pd.DataFrame({
    'ProductID': [101, 102, 103, 104, 105, 106, 107, 111, 112, 113],
    'Company_Location': ['Chennai', 'Mumbai', 'Delhi', 'Bengaluru',
'Hyderabad',
                         'Pune', 'Kolkata', 'Ahmedabad', 'Jaipur', 'Kochi']
})

dataframe3 = pd.DataFrame({
    'ProductID': range(111, 121),
    'NameofProd': ['Speaker', 'Pendrive', 'HardDisk', 'Projector',
'Charger',
```

```
                'PowerBank', 'SmartWatch', 'Earbuds', 'Mic',
'Camera'],
    'Price': [3000, 700, 4500, 32000, 900, 1500, 6000, 2000, 2800,
40000]
})

print("dataframe1:"); display(dataframe1)
print("dataframe2:"); display(dataframe2)
print("dataframe3:"); display(dataframe3)

appended = pd.concat([dataframe1, dataframe3],
ignore_index=True)
print("\n1. Append df1 + df3:"); display(appended)

concat_rows = pd.concat([dataframe1, dataframe3], axis=0,
ignore_index=True)
concat_cols = pd.concat([dataframe1.set_index('ProductID'),
                     dataframe2.set_index('ProductID')],
axis=1).reset_index()
print("\n2a. Row-wise concat (df1, df3):");
display(concat_rows.head())
print("2b. Column-wise concat (df1, df2 aligned on ProductID):");
display(concat_cols)

merged = pd.merge(dataframe1, dataframe2, on='ProductID')
print("\n3. Merge on ProductID:"); display(merged)

inner = pd.merge(dataframe1, dataframe2, on='ProductID',
how='inner')
left  = pd.merge(dataframe1, dataframe2, on='ProductID',
how='left')
right = pd.merge(dataframe1, dataframe2, on='ProductID',
how='right')
outer = pd.merge(dataframe1, dataframe2, on='ProductID',
how='outer')
print("\n4a. Inner Join:"); display(inner)
print("4b. Left Join:");  display(left)
print("4c. Right Join:"); display(right)
print("4d. Full Outer Join:"); display(outer)
```

```
Jeswanth = np.array([212, 45, 78, 120, 9, 300, 56, 150, 33, 97],
dtype=float)
print("\nVector Jeswanth:", Jeswanth)

def rescale(x, new_min, new_max):
    return new_min + (x - x.min()) * (new_max - new_min) / (x.max()
- x.min())

norm_3_3   = rescale(Jeswanth, -3, 3)
norm_1_1   = rescale(Jeswanth, -1, 1)
minmax_0_1 = rescale(Jeswanth, 0, 1)

zscore = (Jeswanth - Jeswanth.mean()) / Jeswanth.std(ddof=1)

j = len(str(int(np.abs(Jeswanth).max())))
decimal_scaled = Jeswanth / (10 ** j)

result = pd.DataFrame({
    'Original': Jeswanth,
    '[-3,+3]': norm_3_3,
    '[-1,+1]': norm_1_1,
    '[0,1] MinMax': minmax_0_1,
    'Z-score': zscore,
    f'Decimal Scaling (/10^{j})': decimal_scaled
})
display(result.round(4))


print("\n\n==================== DA 2
====================")

df = pd.read_csv(DATA)

print("First 10 records:"); display(df.head(10))
print("Last 10 records:");  display(df.tail(10))

print("Rows, Columns:", df.shape)
print("Rows:", df.shape[0], "| Columns:", df.shape[1])
print("\nColumn names:", list(df.columns))
print("\nData types:"); print(df.dtypes)
print(); df.info()
```

```
print("\nSummary statistics:"); display(df.describe(include='all'))

rng = np.random.default_rng(42)
df_na = df.copy()
for c in ['age', 'economic.cond.national',
'economic.cond.household']:
    df_na[c] = df_na[c].astype(float)
for c in ['age', 'economic.cond.national', 'vote', 'gender']:
    idx = rng.choice(df_na.index, size=75, replace=False)
    df_na.loc[idx, c] = np.nan
print("\n(Injected random missing values into a copy for
demonstration)")

print("\nMissing values in each column:"); print(df_na.isna().sum())
print("\nTotal missing values:", int(df_na.isna().sum().sum()))
rows_missing = df_na[df_na.isna().any(axis=1)]
print("\nRows with >=1 missing value:", len(rows_missing));
display(rows_missing.head())
df_dropped = df_na.dropna()
print("\nCleaned dataset (rows with NaN removed):",
df_dropped.shape); display(df_dropped.head())

df_m = df_na.copy()
df_m['age'] = df_m['age'].fillna(df_m['age'].mean())
print("\nMissing in age after mean fill:", df_m['age'].isna().sum())
df_m['economic.cond.national'] =
df_m['economic.cond.national'].fillna(df_m['economic.cond.nation
al'].median())
print("Missing in economic.cond.national after median fill:",
df_m['economic.cond.national'].isna().sum())
df_m['vote'] = df_m['vote'].fillna(df_m['vote'].mode()[0])
print("Missing in vote after mode fill:", df_m['vote'].isna().sum())

df_ff = df_na.ffill()
print("\nAfter forward fill, remaining NaN:",
int(df_ff.isna().sum().sum()))
display(df_ff.head())
df_bf = df_na.bfill()
print("After backward fill, remaining NaN:",
int(df_bf.isna().sum().sum()))
```

```
display(df_bf.head())

df_c = df_ff.copy()
num_c = df_c.select_dtypes(include='number').columns
obj_c = df_c.select_dtypes(exclude='number').columns
df_c[num_c] = df_c[num_c].fillna(0)
df_c[obj_c] = df_c[obj_c].fillna("Unknown")
print("After constant fill, remaining NaN:",
int(df_c.isna().sum().sum()))

cols_wo_id = [c for c in df_c.columns if c != 'rownames']
print("\nDuplicate records:",
df_c.duplicated(subset=cols_wo_id).sum())
df_c = df_c.drop_duplicates(subset=cols_wo_id)
print("Shape after removing duplicates:", df_c.shape)

for c in ['vote', 'gender']:
    df_c[c] = df_c[c].astype('category')
print("\nDtypes after factor conversion:"); print(df_c.dtypes)

df_c = df_c.rename(columns={'economic.cond.national':
'econ_national',
                           'economic.cond.household': 'econ_household',
                           'political.knowledge': 'pol_knowledge'})
print("\nRenamed columns:", list(df_c.columns))

df_c['econ_total'] = df_c['econ_national'] + df_c['econ_household']
display(df_c.head())

print("\nSummary statistics:"); display(df_c.describe(include='all'))

num_cols = ['age', 'econ_national', 'econ_household', 'Blair',
'Hague',
            'Kennedy', 'Europe', 'pol_knowledge', 'econ_total']
print("\nMinimum:"); print(df_c[num_cols].min())
print("\nMaximum:"); print(df_c[num_cols].max())
print("\nRange of age:", df_c['age'].max() - df_c['age'].min())
print("\nVariance:"); print(df_c[num_cols].var())
print("\nStandard deviation:"); print(df_c[num_cols].std())
q1, q3 = df_c['age'].quantile(0.25), df_c['age'].quantile(0.75)
print("\nIQR of age:", q3 - q1, f"(Q1={q1}, Q3={q3})")
```

```
print("IQR of all numeric columns:");
print(df_c[num_cols].quantile(0.75) - df_c[num_cols].quantile(0.25))


print("\n\n==================== DA 3
====================")

df = pd.read_csv(DATA).drop(columns='rownames')

rng = np.random.default_rng(7)
for c in ['age', 'economic.cond.national',
'economic.cond.household', 'Blair']:
    df[c] = df[c].astype(float)
for c in ['age', 'economic.cond.national',
'economic.cond.household', 'Blair', 'vote', 'gender']:
    idx = rng.choice(df.index, size=60, replace=False)
    df.loc[idx, c] = np.nan
df['junk'] = np.where(rng.random(len(df)) < 0.2, 1.0, np.nan)

print("First six:"); display(df.head(6))
print("Last six:");  display(df.tail(6))

print("\nTotal missing values:", int(df.isna().sum().sum()))
miss_rows = df[df.isna().any(axis=1)]
print("Records with >=1 missing value:", len(miss_rows));
display(miss_rows.head())

miss_pct = df.isna().mean() * 100
print("\nMissing % per column:"); print(miss_pct.round(2))
df = df.loc[:, miss_pct <= 70]
print("Columns after dropping >70% missing:", list(df.columns))

num_cols = df.select_dtypes(include='number').columns
df_mean = df.copy()
df_mean[num_cols] =
df_mean[num_cols].fillna(df_mean[num_cols].mean())
print("\nMean-imputed numeric NaNs left:",
int(df_mean[num_cols].isna().sum().sum()))
df_med = df.copy()
df_med[num_cols] =
df_med[num_cols].fillna(df_med[num_cols].median())
```

```
print("Median-imputed numeric NaNs left:",
int(df_med[num_cols].isna().sum().sum()))

df = df_med.copy()
cat_cols = df.select_dtypes(exclude='number').columns
for c in cat_cols:
    df[c] = df[c].fillna(df[c].mode()[0])
print("Total NaNs left:", int(df.isna().sum().sum()))

print("\nDuplicate rows:", df.duplicated().sum())
display(df[df.duplicated(keep=False)].head())
df = df.drop_duplicates().reset_index(drop=True)
print("Shape after removing duplicates:", df.shape)

print("\nCurrent dtypes:"); print(df.dtypes)
print("\nNote:
Blair/Hague/Kennedy/Europe/econ/political.knowledge are ordinal
scores stored as numbers (OK for analysis);"
      "\n      vote & gender are text -> should be categorical.")
for c in ['vote', 'gender']:
    df[c] = df[c].astype('category')
print("\nDtypes after conversion:"); print(df.dtypes)
df['vote'] = df['vote'].astype(str).str.lower().astype('category')
df['gender'] = df['gender'].astype(str).str.upper().astype('category')
print("\nvote (lowercase):", list(df['vote'].cat.categories), "| gender
(uppercase):", list(df['gender'].cat.categories))

plt.figure(figsize=(7, 4))
sns.boxplot(x=df['age'], color='skyblue')
plt.title("Boxplot of Age - Outlier Detection"); plt.xlabel("Age");
plt.show()

q1, q3 = df['age'].quantile(0.25), df['age'].quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = df[(df['age'] < lower) | (df['age'] > upper)]
print(f"\nIQR={iqr}, lower fence={lower}, upper fence={upper},
outliers found={len(outliers)}")
ext_lo, ext_hi = q1 - 3 * iqr, q3 + 3 * iqr
before = len(df)
```

```
df = df[(df['age'] >= ext_lo) & (df['age'] <=
ext_hi)].reset_index(drop=True)
print(f"Extreme outliers removed: {before - len(df)} | rows now:
{len(df)}")

plt.figure(figsize=(7, 4))
plt.hist(df['age'], bins=20, color='steelblue', edgecolor='black')
plt.title("Histogram of Age"); plt.xlabel("Age");
plt.ylabel("Frequency"); plt.show()
sk = skew(df['age'])
shape = "roughly symmetric" if abs(sk) < 0.5 else ("right-skewed"
if sk > 0 else "left-skewed")
print(f"Interpretation: skewness = {sk:.3f} -> distribution is {shape};
mean={df['age'].mean():.1f}, median={df['age'].median():.1f}")

plt.figure(figsize=(7, 4))
plt.hist(df['age'], bins=20, density=True, color='lightblue',
edgecolor='black', alpha=0.7)
sns.kdeplot(df['age'], color='red', linewidth=2)
plt.title("Histogram with Density Curve (Age)"); plt.xlabel("Age");
plt.ylabel("Density"); plt.show()

plt.figure(figsize=(5, 5))
sns.violinplot(y=df['age'], color='lightgreen')
plt.title("Violin Plot of Age"); plt.ylabel("Age"); plt.show()

small = df['age'].sample(30,
random_state=1).sort_values().reset_index(drop=True)
plt.figure(figsize=(8, 3))
plt.plot(small, np.zeros(len(small)), 'o', alpha=0.6, color='purple')
plt.yticks([]); plt.title("Dot Plot of 30 Sampled Ages");
plt.xlabel("Age"); plt.show()

freq = df['vote'].value_counts()
print("\nFrequency table (vote):");
display(freq.to_frame('Frequency'))
prop = df['vote'].value_counts(normalize=True)
print("Proportion & Percentage:")
display(pd.DataFrame({'Proportion': prop.round(4), 'Percentage':
(prop * 100).round(2)}))
```

```
plt.figure(figsize=(6, 4))
sns.barplot(x=freq.index.astype(str), y=freq.values,
palette='viridis')
plt.title("Frequency of Vote"); plt.xlabel("Vote"); plt.ylabel("Count");
plt.show()

eu = df['Europe'].value_counts().sort_index()
plt.figure(figsize=(7, 5))
plt.barh(eu.index.astype(str), eu.values, color='teal')
plt.title("Europe Attitude Score - Frequency (Horizontal)");
plt.xlabel("Count"); plt.ylabel("Europe score"); plt.show()

pct = prop * 100
plt.figure(figsize=(6, 4))
plt.bar(pct.index.astype(str), pct.values, color='coral')
for i, v in enumerate(pct.values):
    plt.text(i, v + 0.5, f"{v:.1f}%", ha='center')
plt.title("Percentage of Vote"); plt.xlabel("Vote");
plt.ylabel("Percentage (%)"); plt.show()

plt.figure(figsize=(6, 6))
plt.pie(freq.values, labels=freq.index.astype(str),
autopct='%1.1f%%', startangle=90)
plt.title("Pie Chart of Vote"); plt.show()

plt.figure(figsize=(6, 4))
x = np.arange(len(freq))
plt.vlines(x, 0, freq.values, color='gray')
plt.plot(x, freq.values, 'o', markersize=10, color='crimson')
plt.xticks(x, freq.index.astype(str))
plt.title("Lollipop Chart of Vote Frequencies"); plt.xlabel("Vote");
plt.ylabel("Count"); plt.show()

plt.figure(figsize=(6, 3))
y = np.arange(len(freq))
plt.hlines(y, 0, freq.values, color='lightgray')
plt.plot(freq.values, y, 'o', color='navy')
plt.yticks(y, freq.index.astype(str))
plt.title("Dot Chart of Vote"); plt.xlabel("Count"); plt.show()

plt.figure(figsize=(7, 4))
```

```
sns.kdeplot(df['age'], fill=True, color='orange')
plt.title("Density Plot of Age"); plt.xlabel("Age"); plt.show()

g = sns.FacetGrid(df, col='vote', col_wrap=3, height=3.5,
sharey=False)
g.map_dataframe(sns.histplot, x='age', kde=True, bins=15,
color='slateblue')
g.set_titles("Vote: {col_name}"); g.set_axis_labels("Age", "Count")
plt.show()
print("\nInterpretation per group:")
for name, grp in df.groupby('vote', observed=True):
    s = skew(grp['age'])
    print(f"  {name}: n={len(grp)}, mean={grp['age'].mean():.1f},
median={grp['age'].median():.1f}, "
          f"std={grp['age'].std():.1f}, skew={s:.2f} -> {'right-skewed' if
s > 0.3 else 'left-skewed' if s < -0.3 else 'roughly symmetric'}")


print("\n\n==================== DA 4
====================")

df = pd.read_csv(DATA)

ct = pd.crosstab(df['vote'], df['gender'])
print("Contingency table (vote x gender):"); display(ct)

ax = ct.plot(kind='bar', stacked=True, figsize=(7, 5),
color=['#e377c2', '#1f77b4'], edgecolor='black')
ax.set_title("Stacked Bar Plot: Vote by Gender (Base)")
ax.set_xlabel("Vote"); ax.set_ylabel("Count");
ax.legend(title="Gender")
plt.xticks(rotation=0); plt.show()

plt.figure(figsize=(7, 5))
sns.histplot(data=df, x='vote', hue='gender', multiple='stack',
shrink=0.8, palette='Set2')
plt.title("Stacked Bar Plot: Vote by Gender (seaborn)")
plt.xlabel("Vote"); plt.ylabel("Count"); plt.show()

plt.figure(figsize=(8, 5))
```

```
sns.countplot(data=df, x='economic.cond.national', hue='vote',
palette='Set1')
plt.title("Grouped Bar: National Economic Rating by Vote")
plt.xlabel("National economic condition (1-5)"); plt.ylabel("Count");
plt.legend(title="Vote"); plt.show()

plt.figure(figsize=(7, 5))
sns.scatterplot(data=df, x='age', y='Europe', alpha=0.4)
plt.title("Scatter Plot: Age vs Europe Attitude Score")
plt.xlabel("Age"); plt.ylabel("Europe score"); plt.show()

trend = df.groupby('age')['Europe'].mean()
plt.figure(figsize=(8, 5))
plt.plot(trend.index, trend.values, marker='o', markersize=3,
color='darkgreen')
plt.title("Line Plot: Mean Europe Score by Age")
plt.xlabel("Age"); plt.ylabel("Mean Europe score"); plt.grid(True);
plt.show()

corr = df[NUM_COLS].corr()
plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm',
linewidths=0.5)
plt.title("Heatmap of Numerical Variable Correlations"); plt.show()

cmp = df.groupby('vote')[['Blair', 'Hague']].mean()
ax = cmp.plot(kind='bar', figsize=(7, 5), edgecolor='black')
ax.set_title("Bar Chart: Mean Blair vs Hague Rating by Vote")
ax.set_xlabel("Vote"); ax.set_ylabel("Mean rating");
ax.legend(title="Leader")
plt.xticks(rotation=0); plt.show()

plt.figure(figsize=(8, 5))
sns.kdeplot(data=df, x='age', hue='vote', fill=True,
common_norm=False, alpha=0.4)
plt.title("Density Plot: Age by Vote"); plt.xlabel("Age");
plt.ylabel("Density"); plt.show()

plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x='vote', y='age', palette='pastel')
```

```
plt.title("Box Plot: Age by Vote"); plt.xlabel("Vote");
plt.ylabel("Age"); plt.show()

plt.figure(figsize=(7, 5))
sns.violinplot(data=df, x='vote', y='age', palette='muted')
plt.title("Violin Plot: Age by Vote"); plt.xlabel("Vote");
plt.ylabel("Age"); plt.show()

plt.figure(figsize=(7, 5))
sns.violinplot(data=df, x='vote', y='age', inner=None,
palette='Set3')
sns.boxplot(data=df, x='vote', y='age', width=0.15,
boxprops={'zorder': 2, 'facecolor': 'white'}, showfliers=False)
plt.title("Violin + Box Plot: Age by Vote"); plt.xlabel("Vote");
plt.ylabel("Age"); plt.show()

corr = df[NUM_COLS].corr()
print("Correlation matrix:"); display(corr.round(3))
mask = np.triu(np.ones_like(corr, dtype=bool))
plt.figure(figsize=(9, 7))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f",
cmap='RdBu_r', vmin=-1, vmax=1,
            square=True, linewidths=0.5, cbar_kws={'label':
'Correlation'})
plt.title("Correlogram of Numerical Variables"); plt.show()


print("\n\n==================== DA 5
====================")

df = pd.read_csv(DATA)
print("Numerical variables  :", NUM_COLS)
print("Categorical variables:", ['vote', 'gender'])
display(df.head())

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='age', y='Europe', hue='vote',
alpha=0.6)
plt.title("Age vs Europe, coloured by Vote"); plt.xlabel("Age");
plt.ylabel("Europe score"); plt.show()
```

```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='age', y='Europe', hue='vote',
style='gender', alpha=0.7, s=60)
plt.title("Age vs Europe: colour = Vote, shape = Gender");
plt.xlabel("Age"); plt.ylabel("Europe score"); plt.show()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='age', y='Europe', hue='vote',
size='political.knowledge',
                sizes=(20, 250), alpha=0.5)
plt.title("Age vs Europe: colour = Vote, size = Political Knowledge")
plt.xlabel("Age"); plt.ylabel("Europe score")
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left'); plt.show()

b = df.groupby('age').agg(Europe=('Europe', 'mean'), Blair=('Blair',
'mean'), n=('age', 'size')).reset_index()
plt.figure(figsize=(9, 6))
plt.scatter(b['age'], b['Europe'], s=b['Blair'] * 60, alpha=0.5,
c=b['Blair'], cmap='viridis', edgecolors='black')
plt.colorbar(label="Mean Blair rating (also bubble size)")
plt.title("Bubble Plot: x=Age, y=Mean Europe score, size=Mean
Blair rating")
plt.xlabel("Age"); plt.ylabel("Mean Europe score"); plt.show()

g = sns.relplot(data=df, x='age', y='Europe', col='vote',
col_wrap=3, height=4, alpha=0.5, hue='vote', legend=False)
g.set_titles("Vote: {col_name}"); g.set_axis_labels("Age", "Europe
score")
g.fig.suptitle("Facet Wrap: Age vs Europe by Vote", y=1.03);
plt.show()

g = sns.FacetGrid(df, row='gender', col='vote', height=3.2,
margin_titles=True)
g.map_dataframe(sns.scatterplot, x='age', y='Europe', alpha=0.5)
g.set_axis_labels("Age", "Europe score")
g.fig.suptitle("Facet Grid: Age vs Europe by Gender (rows) and
Vote (columns)", y=1.03); plt.show()
print("Compare: spread/slope of points across panels shows
whether the age-Europe pattern differs by gender and party.")

pair_cols = ['age', 'Blair', 'Hague', 'Europe', 'political.knowledge']
```

```
sns.pairplot(df[pair_cols + ['vote']], hue='vote', corner=False,
plot_kws={'alpha': 0.4, 's': 20})
plt.suptitle("Pairs Plot of Selected Numerical Variables", y=1.02);
plt.show()
print("Correlations among selected variables:");
display(df[pair_cols].corr().round(3))

pc = df[pair_cols + ['vote']].sample(200, random_state=42).copy()
pc[pair_cols] = (pc[pair_cols] - pc[pair_cols].mean()) /
pc[pair_cols].std()
plt.figure(figsize=(10, 6))
parallel_coordinates(pc, 'vote', color=sns.color_palette("Set1",
pc['vote'].nunique()), alpha=0.4)
plt.title("Parallel Coordinate Plot (standardised), coloured by Vote")
plt.ylabel("Standardised value"); plt.xticks(rotation=20);
plt.legend(title="Vote"); plt.show()


print("\n\n==================== DA 6
====================")

vals = [
 112,118,132,129,121,135,148,148,136,119,104,118,
 115,126,141,135,125,149,170,170,158,133,114,140,
 145,150,178,163,172,178,199,199,184,162,146,166,
 171,180,193,181,183,218,230,242,209,191,172,194,
 196,196,236,235,229,243,264,272,237,211,180,201,
 204,188,235,227,234,264,302,293,259,229,203,229,
 242,233,267,269,270,315,364,347,312,274,237,278,
 284,277,317,313,318,374,413,405,355,306,271,306,
 315,301,356,348,355,422,465,467,404,347,305,336,
 340,318,362,348,363,435,491,505,404,359,310,337,
 360,342,406,396,420,472,548,559,463,407,362,405,
 417,391,419,461,472,535,622,606,508,461,390,432]
AirPassengers = pd.Series(vals, index=pd.date_range("1949-01-
01", periods=144, freq="MS"), name="Passengers")

print("First 5:"); print(AirPassengers.head(5))
print("\nLast 5:");  print(AirPassengers.tail(5))

print("\nStart:", AirPassengers.index[0].strftime("%b %Y"))
```

```
print("End  :", AirPassengers.index[-1].strftime("%b %Y"))

print("\nFrequency (observations per year): 12  | pandas freq:",
AirPassengers.index.freqstr)
print("Class:", type(AirPassengers))
print("Length:", len(AirPassengers))
display(AirPassengers.describe())

plt.figure(figsize=(11, 4))
plt.plot(AirPassengers, color='navy')
plt.title("Monthly Air Passengers (1949-1960)"); plt.xlabel("Year");
plt.ylabel("Passengers (thousands)")
plt.grid(True); plt.show()

decomp = seasonal_decompose(AirPassengers, model='additive',
period=12)
fig = decomp.plot(); fig.set_size_inches(11, 8);
plt.suptitle("Decomposition of AirPassengers (additive)", y=1.02);
plt.show()
print("Seasonal figures (first 12):");
print(decomp.seasonal.head(12).round(2))

fig, ax = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(AirPassengers, lags=36, ax=ax[0]); ax[0].set_title("ACF")
plot_pacf(AirPassengers, lags=36, ax=ax[1], method='ywm');
ax[1].set_title("PACF")
plt.show()

model = ARIMA(AirPassengers, order=(0, 1, 1), seasonal_order=(0,
1, 1, 12))
fit = model.fit()
print(fit.summary())
print("\nInterpretation: ma.L1 = non-seasonal MA(1) coefficient;
ma.S.L12 = seasonal MA coefficient;"
      "\nd=1 and seasonal D=1 remove trend and yearly seasonality.
Lower AIC/BIC = better fit;"
      f"\nAIC = {fit.aic:.2f}, BIC = {fit.bic:.2f}.")

fc = fit.get_forecast(steps=12)
fc_mean = fc.predicted_mean
fc_ci = fc.conf_int()
```

```
print("\n12-month forecast:"); print(fc_mean.round(1))

plt.figure(figsize=(11, 5))
plt.plot(AirPassengers, label="Actual", color='navy')
plt.plot(fc_mean, label="Forecast", color='red')
plt.fill_between(fc_ci.index, fc_ci.iloc[:, 0], fc_ci.iloc[:, 1],
color='red', alpha=0.2, label="95% CI")
plt.title("AirPassengers: Actual vs 12-Month ARIMA Forecast")
plt.xlabel("Year"); plt.ylabel("Passengers (thousands)"); plt.legend();
plt.grid(True); plt.show()


print("\n\n==================== DA 7
====================")

df = pd.read_csv(DATA)

x = df['age']
summary_df = pd.DataFrame({
    'Mean': [x.mean()], 'Median': [x.median()], 'Minimum': [x.min()],
'Maximum': [x.max()],
    'Range': [x.max() - x.min()], 'Variance': [x.var()], 'Std Dev':
[x.std()],
    'IQR': [x.quantile(0.75) - x.quantile(0.25)],
    'Skewness': [skew(x)], 'Kurtosis': [kurtosis(x)]
}, index=['age']).T
print("Summary statistics of age:"); display(summary_df.round(4))

freq = df['vote'].value_counts()
rel = df['vote'].value_counts(normalize=True)
print("\nFrequency distribution & relative frequency (vote):")
display(pd.DataFrame({'Frequency': freq, 'Relative Frequency':
rel.round(4)}))
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].pie(freq.values, labels=freq.index, autopct='%1.1f%%',
startangle=90); ax[0].set_title("Pie Chart: Vote")
ax[1].bar(freq.index, freq.values, color='steelblue',
edgecolor='black')
ax[1].set_title("Bar Plot: Vote"); ax[1].set_xlabel("Vote");
ax[1].set_ylabel("Count")
plt.show()
```

```
bins = pd.cut(df['age'], bins=8)
bt = bins.value_counts().sort_index()
print("\nFrequency distribution with bin intervals (age):")
display(pd.DataFrame({'Frequency': bt, 'Relative Frequency': (bt /
bt.sum()).round(4)}))
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].hist(df['age'], bins=8, color='skyblue', edgecolor='black');
ax[0].set_title("Histogram of Age")
ax[0].set_xlabel("Age"); ax[0].set_ylabel("Frequency")
sns.boxplot(x=df['age'], ax=ax[1], color='lightgreen');
ax[1].set_title("Box Plot of Age (outlier check)")
plt.show()

ct = pd.crosstab(df['vote'], df['gender'])
print("\nContingency table:"); display(ct)
print("Row percentages (%):");    display((pd.crosstab(df['vote'],
df['gender'], normalize='index') * 100).round(2))
print("Column percentages (%):"); display((pd.crosstab(df['vote'],
df['gender'], normalize='columns') * 100).round(2))

fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ct.plot(kind='bar', stacked=True, ax=ax[0], edgecolor='black');
ax[0].set_title("Stacked Bar: Vote by Gender")
ct.plot(kind='bar', ax=ax[1], edgecolor='black');
ax[1].set_title("Grouped Bar: Vote by Gender")
for a in ax: a.set_xlabel("Vote"); a.set_ylabel("Count");
a.tick_params(axis='x', rotation=0)
plt.show()

agg = df.groupby('vote')['age'].agg(['mean', 'median',
'std']).round(3)
print("\nAge statistics by vote:"); display(agg)

fig, ax = plt.subplots(1, 2, figsize=(13, 5))
sns.boxplot(data=df, x='vote', y='age', ax=ax[0], palette='pastel');
ax[0].set_title("Side-by-side Box Plots: Age by Vote")
sns.violinplot(data=df, x='vote', y='age', ax=ax[1],
palette='muted'); ax[1].set_title("Violin Plot: Age by Vote")
plt.show()
```

```
pear = df['Blair'].corr(df['Europe'], method='pearson')
spear = df['Blair'].corr(df['Europe'], method='spearman')
print(f"\nPearson r (Blair, Europe)  = {pear:.4f}")
print(f"Spearman rho (Blair, Europe) = {spear:.4f}")
plt.figure(figsize=(7, 5))
sns.regplot(data=df, x='Blair', y='Europe', x_jitter=0.2, y_jitter=0.2,
            scatter_kws={'alpha': 0.3}, line_kws={'color': 'red'})
plt.title("Blair vs Europe with Linear Trend Line"); plt.xlabel("Blair
rating"); plt.ylabel("Europe score"); plt.show()

stats2 = df.groupby(['vote', 'gender'])['age'].agg(['mean',
'std']).round(3)
print("\nMean & SD of age by vote x gender:"); display(stats2)
sns.catplot(data=df, x='vote', y='age', col='gender', kind='box',
height=4.5, aspect=1)
plt.suptitle("Faceted Box Plot: Age by Vote, split by Gender",
y=1.03); plt.show()
means = df.groupby(['vote', 'gender'])['age'].mean().unstack()
sds = df.groupby(['vote', 'gender'])['age'].std().unstack()
ax = means.plot(kind='bar', yerr=sds, capsize=4, figsize=(8, 5),
edgecolor='black')
ax.set_title("Grouped Bar Chart with Error Bars (mean +/- SD of
Age)")
ax.set_xlabel("Vote"); ax.set_ylabel("Mean age");
plt.xticks(rotation=0); plt.show()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='age', y='Europe', hue='vote',
style='vote', alpha=0.6, s=50)
plt.title("Age vs Europe by Vote (colour & shape)"); plt.show()
for name, g in df.groupby('vote'):
    print(f"\nCorrelation matrix (age, Europe) for vote = {name}:")
    display(g[['age', 'Europe']].corr().round(3))

corr = df[NUM_COLS].corr()
plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm',
linewidths=0.5)
plt.title("Correlation Matrix Heatmap"); plt.show()
```

```
sns.pairplot(df[['age', 'Blair', 'Hague', 'Europe',
'political.knowledge']], diag_kind='kde',
            plot_kws={'alpha': 0.3, 's': 15})
plt.suptitle("Pairs Plot / Scatterplot Matrix", y=1.02); plt.show()


print("\n\n==================== DA 8
====================")

df = pd.read_csv(DATA)
display(df.head())
print("Shape:", df.shape)

print("\nMissing values:"); print(df.isna().sum())

print("\nDistribution of vote:"); print(df['vote'].value_counts())

X = df[NUM_COLS].copy()
print("\nFeatures used:", list(X.columns))

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
display(X_scaled_df.head())

km2 = KMeans(n_clusters=2, n_init=25, random_state=42)
km2.fit(X_scaled)
print("\nK-Means (K=2) cluster sizes:", np.bincount(km2.labels_))
print("Within-cluster SS (inertia):", round(km2.inertia_, 2))
print("First 15 cluster labels:", km2.labels_[:15])

print("\nCluster centers:")
display(pd.DataFrame(km2.cluster_centers_,
columns=X.columns).round(3))

wcss = []
for k in range(1, 11):
    wcss.append(KMeans(n_clusters=k, n_init=25,
random_state=42).fit(X_scaled).inertia_)
display(pd.DataFrame({'K': range(1, 11), 'WCSS': np.round(wcss,
2)}))
```

```
plt.figure(figsize=(7, 5))
plt.plot(range(1, 11), wcss, 'bo-')
plt.title("Elbow Method for Optimal K"); plt.xlabel("Number of
clusters (K)"); plt.ylabel("WCSS")
plt.xticks(range(1, 11)); plt.grid(True); plt.show()

k_opt = int(np.argmax(np.diff(wcss, 2))) + 2
print(f"Elbow recommendation: K = {k_opt}")
km_opt = KMeans(n_clusters=k_opt, n_init=25,
random_state=42).fit(X_scaled)
print("Cluster sizes:", np.bincount(km_opt.labels_))
display(pd.DataFrame(km_opt.cluster_centers_,
columns=X.columns).round(3))
print("Clusters vs actual vote:")
display(pd.crosstab(km_opt.labels_, df['vote'],
rownames=['Cluster'], colnames=['Vote']))

df_h = pd.read_csv(DATA)
display(df_h.head())
print("\nMissing values:", int(df_h.isna().sum().sum()))

Xh = df_h[NUM_COLS]
Xh_scaled = StandardScaler().fit_transform(Xh)

d = pdist(Xh_scaled, metric='euclidean')
print("Number of pairwise distances:", len(d))

Z = linkage(d, method='ward')

plt.figure(figsize=(12, 6))
dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=90,
leaf_font_size=9, show_contracted=True)
plt.title("Hierarchical Clustering Dendrogram (Ward's method,
truncated)")
plt.xlabel("Cluster (size in brackets)"); plt.ylabel("Distance");
plt.show()

clusters = fcluster(Z, t=3, criterion='maxclust')
print("\nFirst 20 cluster assignments:", clusters[:20])
```

```
print("\nNumber of samples in each cluster:")
print(pd.Series(clusters).value_counts().sort_index())


print("\n\n==================== DA 9
====================")

df = pd.read_csv(DATA)
print("Libraries ready: seaborn/matplotlib (ggplot2), sklearn PCA
(FactoMineR/factoextra)")

print("Total rows:", df.shape[0], "| Total columns:", df.shape[1])

num_df = df[NUM_COLS].dropna()
data_scaled =
pd.DataFrame(StandardScaler().fit_transform(num_df),
columns=num_df.columns, index=num_df.index)
display(data_scaled.head())

pca = PCA()
scores = pca.fit_transform(data_scaled)
pc_names = [f"PC{i+1}" for i in range(scores.shape[1])]
summ = pd.DataFrame({
    'Standard deviation': np.sqrt(pca.explained_variance_),
    'Proportion of Variance': pca.explained_variance_ratio_,
    'Cumulative Proportion':
np.cumsum(pca.explained_variance_ratio_)
}, index=pc_names).round(4)
print("\nPCA summary:"); display(summ)

var_pct = pca.explained_variance_ratio_ * 100
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(var_pct) + 1), var_pct, 'o-', color='steelblue')
plt.bar(range(1, len(var_pct) + 1), var_pct, alpha=0.4,
color='steelblue')
for i, v in enumerate(var_pct):
    plt.text(i + 1, v + 1, f"{v:.1f}%", ha='center')
plt.ylim(0, 60)
plt.title("Scree Plot - Percentage of Variance Explained")
plt.xlabel("Principal Component"); plt.ylabel("Percentage of
variance (%)")
```

```
plt.xticks(range(1, len(var_pct) + 1)); plt.show()

loadings = pca.components_.T
scale = np.abs(scores[:, :2]).max()
fig, ax = plt.subplots(figsize=(10, 8))
ax.scatter(scores[:, 0], scores[:, 1], s=8, alpha=0.25, color='grey')
texts = []
for i, f in enumerate(data_scaled.columns):
    ax.arrow(0, 0, loadings[i, 0] * scale * 0.8, loadings[i, 1] * scale *
0.8,
             color='red', alpha=0.8, head_width=0.08,
length_includes_head=True)
    texts.append(ax.text(loadings[i, 0] * scale * 0.85, loadings[i, 1] *
scale * 0.85, f,
                         color='darkred', fontsize=10))
adjust_text(texts, ax=ax)
ax.axhline(0, color='black', lw=0.5); ax.axvline(0, color='black',
lw=0.5)
ax.set_title("PCA Biplot (PC1 vs PC2)")
ax.set_xlabel(f"PC1 ({var_pct[0]:.1f}%)"); ax.set_ylabel(f"PC2
({var_pct[1]:.1f}%)")
plt.show()

pca_scores = pd.DataFrame(scores, columns=pc_names,
index=data_scaled.index)
print("\nFirst six rows of pca_scores:");
display(pca_scores.head(6))

plt.figure(figsize=(8, 6))
plt.scatter(pca_scores['PC1'], pca_scores['PC2'], color='blue',
marker='o', alpha=0.6, edgecolors='none')
plt.title("PC1 vs PC2 Scatter Plot - 23BDS0212")
plt.xlabel("PC1"); plt.ylabel("PC2"); plt.grid(True); plt.show()

import os, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from IPython.display import display
from scipy import stats
import statsmodels.api as sm

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)

DATA = "BEPS.csv"
if not os.path.exists(DATA):
    from google.colab import files
    uploaded = files.upload()
    DATA = list(uploaded.keys())[0]

print("==================== DA 10
====================")

df = pd.read_csv(DATA)
print("Shape:", df.shape)
display(df.head())

Xv, Yv = 'Hague', 'Europe'

rng = np.random.default_rng(10)
df[[Xv, Yv]] = df[[Xv, Yv]].astype(float)
for c in [Xv, Yv]:
    idx = rng.choice(df.index, size=40, replace=False)
    df.loc[idx, c] = np.nan

print("\nMissing values per column:"); print(df.isna().sum())
print("Total missing:", int(df.isna().sum().sum()))
for c in [Xv, Yv]:
    df[c] = df[c].fillna(df[c].median())
print("Total missing after median imputation:",
int(df.isna().sum().sum()))

cols_wo_id = [c for c in df.columns if c != 'rownames']
print("\nDuplicate records:",
df.duplicated(subset=cols_wo_id).sum())
df =
df.drop_duplicates(subset=cols_wo_id).reset_index(drop=True)
```

```
print("Shape after removing duplicates:", df.shape)

def iqr_outliers(s):
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    i = q3 - q1
    lo, hi = q1 - 1.5 * i, q3 + 1.5 * i
    return lo, hi, s[(s < lo) | (s > hi)]

for c in [Xv, Yv]:
    lo, hi, out = iqr_outliers(df[c])
    print(f"\n{c}: lower fence={lo:.2f}, upper fence={hi:.2f},
outliers={len(out)}")

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].boxplot(df[Xv]); ax[0].set_title(f"Boxplot of {Xv} (X)");
ax[0].set_ylabel(Xv)
ax[1].boxplot(df[Yv]); ax[1].set_title(f"Boxplot of {Yv} (Y)");
ax[1].set_ylabel(Yv)
plt.show()

print(f"\nIndependent variable X = {Xv}")
print(f"Dependent variable   Y = {Yv}")

desc = pd.DataFrame({
    'Mean': [df[Xv].mean(), df[Yv].mean()],
    'Median': [df[Xv].median(), df[Yv].median()],
    'Variance': [df[Xv].var(), df[Yv].var()],
    'Std Dev': [df[Xv].std(), df[Yv].std()]
}, index=[Xv, Yv])
display(desc.round(4))

plt.figure(figsize=(7, 4))
plt.hist(df[Xv], bins=5, color='skyblue', edgecolor='black')
plt.title(f"Histogram of {Xv} (hist)"); plt.xlabel(Xv);
plt.ylabel("Frequency"); plt.show()

plt.figure(figsize=(7, 4))
sns.histplot(df[Xv], bins=5, color='steelblue')
plt.title(f"Histogram of {Xv} (seaborn)"); plt.xlabel(Xv);
plt.ylabel("Count"); plt.show()
```

```
plt.figure(figsize=(7, 4))
sns.histplot(df[Yv], bins=11, color='darkorange')
plt.title(f"Histogram of {Yv}"); plt.xlabel(Yv); plt.ylabel("Count");
plt.show()

plt.figure(figsize=(7, 4))
sns.kdeplot(df[Xv], fill=True, label=Xv, bw_adjust=1.5)
sns.kdeplot(df[Yv], fill=True, label=Yv, bw_adjust=1.5)
plt.title("Density Plots of X and Y"); plt.xlabel("Value");
plt.ylabel("Density"); plt.legend(); plt.show()

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
sns.boxplot(y=df[Xv], ax=ax[0], color='lightblue');
ax[0].set_title(f"Boxplot of {Xv}")
sns.boxplot(y=df[Yv], ax=ax[1], color='lightgreen');
ax[1].set_title(f"Boxplot of {Yv}")
plt.show()

plt.figure(figsize=(6, 5))
plt.plot(df[Xv], df[Yv], 'o', alpha=0.3, color='navy')
plt.title(f"Scatter Plot: {Xv} vs {Yv} (plot)"); plt.xlabel(Xv);
plt.ylabel(Yv); plt.show()

plt.figure(figsize=(6, 5))
sns.scatterplot(x=df[Xv], y=df[Yv], alpha=0.3)
plt.title(f"Scatter Plot: {Xv} vs {Yv} (scatterplot)"); plt.xlabel(Xv);
plt.ylabel(Yv); plt.show()

r, p = stats.pearsonr(df[Xv], df[Yv])
print(f"\nPearson correlation r = {r:.4f} (p-value = {p:.4g})")
print("Strength:", "weak" if abs(r) < 0.3 else "moderate" if abs(r) <
0.7 else "strong",
      "| Direction:", "positive" if r > 0 else "negative")

plt.figure(figsize=(6, 5))
sns.regplot(x=df[Xv], y=df[Yv], x_jitter=0.15, y_jitter=0.15,
            scatter_kws={'alpha': 0.3}, line_kws={'color': 'red'})
plt.title(f"{Yv} vs {Xv} with Linear Regression Line"); plt.xlabel(Xv);
plt.ylabel(Yv); plt.show()

rs = np.random.RandomState(123)
```

```
perm = rs.permutation(len(df))
n_train = int(0.8 * len(df))
train = df.iloc[perm[:n_train]]
test = df.iloc[perm[n_train:]]
print("\nTraining set:", train.shape, "| Testing set:", test.shape)

X_train = sm.add_constant(train[[Xv]])
model = sm.OLS(train[Yv], X_train).fit()

print(model.summary())

intercept = model.params['const']
slope = model.params[Xv]
print(f"\nIntercept (b0) = {intercept:.4f}")
print(f"Slope (b1)     = {slope:.4f}")
print(f"\nRegression equation: {Yv} = {intercept:.4f} + ({slope:.4f}) x
{Xv}")
print(f"Interpretation: when {Xv} is 0, predicted {Yv} is
{intercept:.2f}; "
      f"each 1-unit increase in {Xv} changes predicted {Yv} by
{slope:.4f}.")

print("\nP-values of coefficients:")
display(model.pvalues.to_frame('p-value'))
sig = "statistically significant" if model.pvalues[Xv] < 0.05 else "not
statistically significant"
print(f"Slope p-value = {model.pvalues[Xv]:.4g} -> {sig} at the 5%
level")

print(f"\nR-squared          = {model.rsquared:.4f}")
print(f"Adjusted R-squared = {model.rsquared_adj:.4f}")
print(f"The model explains about {model.rsquared * 100:.2f}% of
the variation in {Yv}.")

X_test = sm.add_constant(test[[Xv]], has_constant='add')
pred = model.predict(X_test)
actual = test[Yv]
residuals = actual - pred

results = pd.DataFrame({'Actual': actual, 'Predicted':
pred.round(4), 'Residual': residuals.round(4)})
```

```
print("\nFirst 10 test predictions and residuals:");
display(results.head(10))

mae = np.mean(np.abs(residuals))
mse = np.mean(residuals ** 2)
rmse = np.sqrt(mse)
nz = actual != 0
mape = np.mean(np.abs(residuals[nz] / actual[nz])) * 100

print(f"\nMAE  = {mae:.4f}")
print(f"MSE  = {mse:.4f}")
print(f"RMSE = {rmse:.4f}")
print(f"MAPE = {mape:.2f}% (excluded {int((~nz).sum())} zero-
actual observations)")

plt.figure(figsize=(6, 5))
plt.scatter(test[Xv], actual, alpha=0.3, label="Actual (test)")
xs = np.linspace(df[Xv].min(), df[Xv].max(), 100)
plt.plot(xs, intercept + slope * xs, color='red', label="Fitted line")
plt.title("Fitted Regression Line on Test Data"); plt.xlabel(Xv);
plt.ylabel(Yv); plt.legend(); plt.show()
```